

Querying Temporal Data

Boris Novikov

Independent researcher - Helsinki, Finland

Slides and Examples:

Prague, 23 June 2025



Quick Intro

*

*

*

*

*

Defining Temporal Databases

Shameless Ads

- Temporal is NOT temporary
- **Queries to a temporal database return facts that were, are, or will be actual at a time that may differ from current time**
- More details in the introductory articles:
 - Saving Data Historically with Temporal Tables: Part 1: Queries
<https://www.red-gate.com/simple-talk/databases/postgresql/saving-data-historically-with-temporal-tables-part-1-queries/#comments>
 - Making Temporal Databases Work. Part 2: Computing Aggregates Across Temporal Versions
<https://www.red-gate.com/simple-talk/databases/postgresql/making-temporal-databases-work-part-2-computing-aggregates-across-temporal-versions/>
 - Making Temporal Databases Work. Part 3: Saving Data
<https://www.red-gate.com/simple-talk/databases/postgresql/making-temporal-databases-work-part-3-saving-data/>
- B. Novikov and E. Gorshkova. Temporal databases: From theory to applications. Programming and Computer Software, 34(1):1–6, **2008**.
URL: <https://link.springer.com/article/10.1134/S0361768808010015>.



Temporal Database Models

Capturing Real World Changes

A model for this talk:

- Each data row has an associated time period
- Periods are closed-open: [. . .) [.)
- Objects are identified with ***business keys***
 - Rows are identified with compound key (business key and period), but may have surrogate ID
 - Periods for same business key cannot overlap (at most one value for any point in time)
- The model is NOT relational as the OVERLAPS predicate is not an equivalence

Semantics of Time

Time Dimensions

- System (transactional)
 - time when the database was updated
- Valid (effective, application)
 - when facts were valid in the reality
- Belief (asserted)
 - when users believe the facts are valid
- Dozens more are described in the research literature
- Implementations usually start with system time
- we are using valid time in this talk

Table Definitions

Converting current-state DB into temporal

- Adding valid time ranges and integrity constraints to tables in non-temporal version of the database

```
create table temporal_emp (  
  emp_no      int  not null, /* Business key */  
  name        text  not null,  
  salary      int  not null,  
  project     text  not null,  
  emp_row_id bigint not null primary key  
    generated by default as identity,  
  valid_at    tstzrange not null,  
  — SQL Standard syntax  
  /* CONSTRAINT emp_pk PRIMARY KEY  
    (emp_no, valid_at WITHOUT OVERLAPS) */  
  — PostgreSQL 17 syntax  
  EXCLUDE USING GIST (emp_no WITH =, valid_at WITH &&)  
);
```

```
create table temporal_project (  
  proj_code   text not null,  
  proj_title  text not null,  
  proj_status text not null,  
  proj_row_id bigint not null primary key  
    generated by default as identity,  
  valid_at    tstzrange not null,  
  EXCLUDE USING GIST  
  (proj_code WITH =, valid_at WITH &&)  
);  
drop table if exists qu;  
create table qu (  
  code       text not null,  
  valid_at   tstzrange not null,  
  EXCLUDE USING GIST (code WITH =, valid_at WITH &&)  
);
```

Populating Temporal Tables 1

temporal_emp

emp_no	name	salary	project	emp_row_id	valid_at
1	Anne	4000	p11	1	["2024-01-01 00:00:00+02", "2024-05-01 00:00:00+03")
1	Anne	4000	p15	2	["2024-05-01 00:00:00+03", infinity)
2	Esa	3200	p20	3	["2024-01-01 00:00:00+02", "2024-04-16 00:00:00+03")
2	Esa	3600	p11	4	["2024-04-16 00:00:00+03", "2024-08-01 00:00:00+03")
3	Eja	4500	p15	5	["2024-01-01 00:00:00+02", "2024-04-20 00:00:00+03")
3	Eja	4800	p15	6	["2024-04-20 00:00:00+03", infinity)
4	Timo	4800	p15	7	["2024-01-01 00:00:00+02", "2024-06-15 00:00:00+03")
4	Timo	4800	p20	8	["2024-06-15 00:00:00+03", "2024-07-01 00:00:00+03")
4	Timo	5000	p20	9	["2024-07-01 00:00:00+03", infinity)
5	Antti	3600	p15	10	["2024-06-01 00:00:00+03", "2024-09-01 00:00:00+03")
5	Antti	3900	p15	11	["2024-09-01 00:00:00+03", "2025-01-01 00:00:00+02")
6	Merja	4200	p11	12	["2024-01-01 00:00:00+02", "2024-03-01 00:00:00+02")
6	Merja	4200	p15	13	["2024-03-01 00:00:00+02", "2024-07-01 00:00:00+03")
6	Merja	4200	p20	14	["2024-07-01 00:00:00+03", "2024-09-01 00:00:00+03")
6	Merja	4800	p20	15	["2024-09-01 00:00:00+03", infinity)

Populating Temporal Tables 2

temporal_project and qu (quarter periods)

proj_code	proj_title	proj_status	proj_row_id	valid_at
p11	temporal join	active	4	["2024-01-01 00:00:00+02","2024-08-01 00:00:00+03")
p11	temporal join	closed	7	["2024-08-01 00:00:00+03",infinity)
p15	temporal aggregates	active	5	["2024-01-01 00:00:00+02","2025-01-01 00:00:00+02")
p15	temporal aggregates	closed	8	["2025-01-01 00:00:00+02",infinity)
p20	temporal update	active	6	["2024-01-01 00:00:00+02","2025-01-01 00:00:00+02")
p20	temporal update	closed	9	["2025-01-01 00:00:00+02",infinity)
p24	temporal FK	active	10	["2024-10-01 00:00:00+03","2025-04-01 00:00:00+03")
p24	temporal FK	closed	11	["2025-04-01 00:00:00+03",infinity)

(8 rows)

code	valid_at
23.q1	["2024-01-01 00:00:00+02","2024-04-01 00:00:00+03")
23.q2	["2024-04-01 00:00:00+03","2024-07-01 00:00:00+03")
23.q3	["2024-07-01 00:00:00+03","2024-10-01 00:00:00+03")
23.q4	["2024-10-01 00:00:00+03","2025-01-01 00:00:00+02")
24.q1	["2025-01-01 00:00:00+02","2025-04-01 00:00:00+03")
24.q2	["2025-04-01 00:00:00+03","2025-07-01 00:00:00+03")
24.q3	["2025-07-01 00:00:00+03","2025-10-01 00:00:00+03")
24.q4	["2025-10-01 00:00:00+03","2026-01-01 00:00:00+02")

(8 rows)

Quick Intro Getting Data from a Temporal Database

*

*

*



Classes of Queries

What Can the User Get from a Temporal Database?

- Time Machine: Point-in-time queries (special case: current time)
- Event logs: History of an item (row, object, attribute, etc.)
- Fully temporal: produce temporal result
 - Output can be used as an input to other temporal queries
 - Relate co-existing objects without explicit specification of time period

Point-in-Time Queries

Time Travel

```
psql> select emp_no, name, project, salary
      from temporal_emp
      where valid_at @> '2024-06-12'::timestamptz;
```

emp_no	name	project	salary
1	Anne	p15	4000
2	Esa	p11	3600
3	Eja	p15	4800
4	Timo	p15	4800
5	Antti	p15	3600
6	Merja	p15	4200

(6 rows)

Querying History of an object

Event Logs

```
psql> select emp_no, name, project, salary, emp_row_id,
  - Intersection
  valid_at * '["2024-04-01", "2024-10-01")'::tstzrange as valid_at
from temporal_emp
where name = 'Timo'
  - Overlaps
  and valid_at && '["2024-04-01", "2024-10-01")'::tstzrange
;
emp_no | name | project | salary | emp_row_id | valid_at
-----+-----+-----+-----+-----+-----
      4 | Timo | p15      |  4800 |          7 | ["2024-04-01", "2024-06-15")
      4 | Timo | p20      |  4800 |          8 | ["2024-06-15", "2024-07-01")
      4 | Timo | p20      |  5000 |          9 | ["2024-07-01", "2024-10-01")
(3 rows)
```

Fully Temporal Queries

Temporal Inner Join

```
psql> select e.project,  p.proj_title,  e.salary,
        e.valid_at * p.valid_at as valid_at
        from temporal_emp e
        join temporal_project p
        on e.project = p.proj_code
           and e.valid_at && p.valid_at
        ;
```

project	proj_title	salary	valid_at
p11	temporal join	4000	["2024-01-01 00:00:00+02","2024-05-01 00:00:00+03")
p15	temporal aggregates	4000	["2025-01-01 00:00:00+02",infinity)
p15	temporal aggregates	4000	["2024-05-01 00:00:00+03","2025-01-01 00:00:00+02")
p20	temporal update	3200	["2024-01-01 00:00:00+02","2024-04-16 00:00:00+03")
p11	temporal join	3600	["2024-04-16 00:00:00+03","2024-08-01 00:00:00+03")
p15	temporal aggregates	4500	["2024-01-01 00:00:00+02","2024-04-20 00:00:00+03")
p15	temporal aggregates	4800	["2025-01-01 00:00:00+02",infinity)
p15	temporal aggregates	4800	["2024-04-20 00:00:00+03","2025-01-01 00:00:00+02")
. . .			

The Real World is Changing

Modifying Temporal Data

- Nothing is deleted from temporal tables, periods are closed instead.
- Updating a column value (temporal delete followed by insert):

```
with temporal_delete as (  
  update temporal_emp set valid_at = '2024-10-01'  
  where name = 'Timo'  
        and valid_at @> '2024-10-01'::timestampz  
  returning *)  
insert into temporal_emp  
  (emp_no, name, salary, project, valid_at)  
select  
  emp_no, name, salary, 'p24',  
  tstzrange(upper(valid_at), 'infinity')  
from temporal_delete;
```

Quick Intro
Getting Data from a Temporal Database

Integrity Constraints

*

*



Making Constraints Temporal

- Additional constraint: Periods for same business key cannot overlap
 - UNIQUE (business_key, period) is not enough
- NOT NULL, CHECK, UNIQUE are easy but should include periods
- All constraints are easy to check for system time
- FOREIGN KEY is expensive (checking of multiple periods)

```
CONSTRAINT fk_constraint FOREIGN KEY fk REFERENCES (ref_table(RK))
```

For INSERT/UPDATE of fk Is

```
(Select range_agg(period) from fk_table where fk=NEW_FK_VALUE) <<  
(Select range_agg(ref_k) from ref_table where ref_k = NEW_FK_VALUE)
```

And for UPDATE/DELET of ref_k is

```
(Select range_agg(period) from fk_table where fk=OLD_REF_K_VALUE) <<  
(Select range_agg(ref_k) from ref_table where ref_k = OLD_REF_K_VALUE)
```

Quick Intro
Getting Data from a Temporal Database
Integrity Constraints

Temporal Aggregation

*

*



Point-in-Time vs. Period Semantics

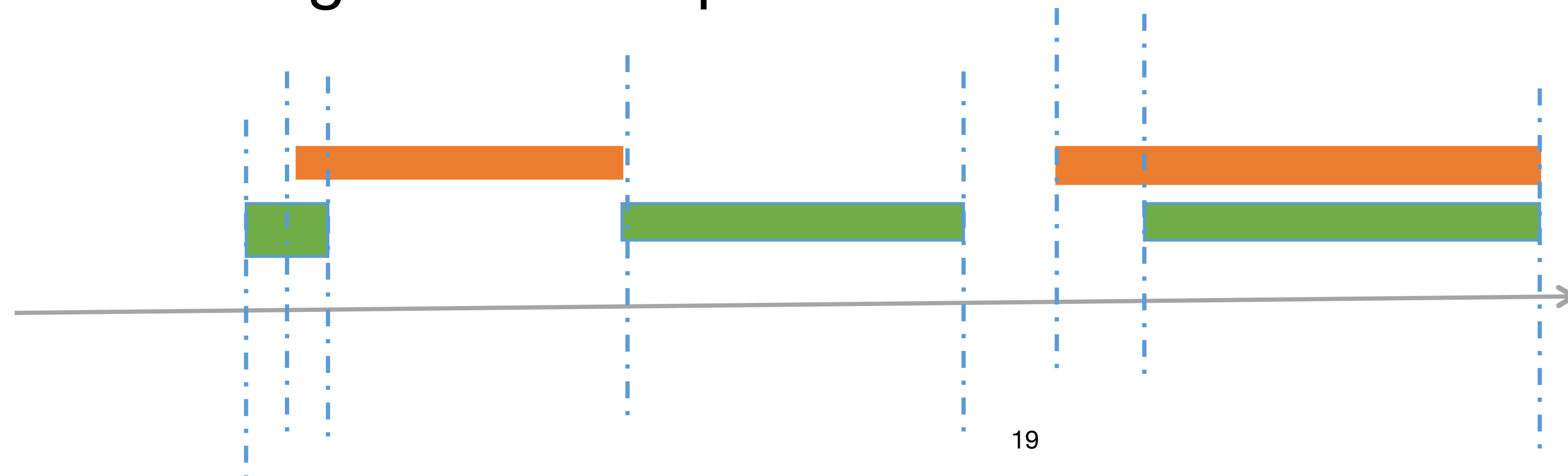
What the Time Period Means?

- Point-in-Time ensures correctness of the time machine: values are same as obtained from non-temporal DB at given point in time
- Period: values are associated with a period
- Point-in-time aggregated values change when any of aggregated values changes
- Period semantics is needed to make aggregation useful

Alignment Operator

Reducing the Temporal Algebra to Relational

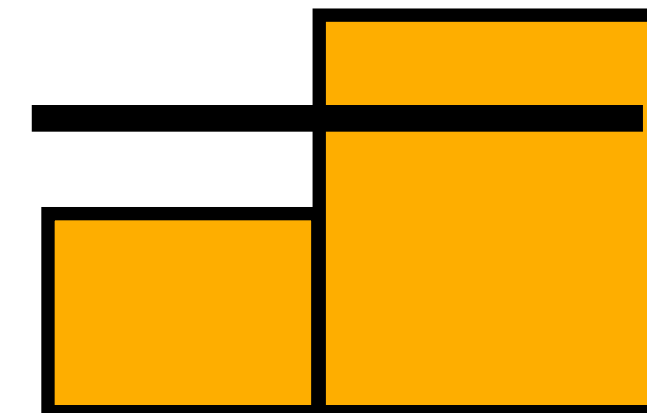
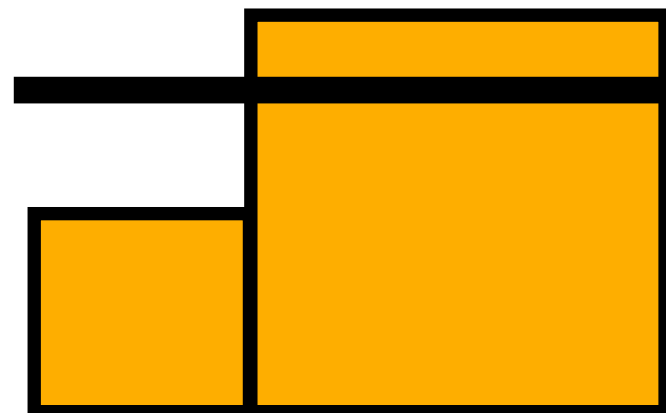
- Alignment operator splits all periods (duplicating other attributes) so that any periods either coincide or do not overlap.
- The OVERLAPS predicate becomes an equivalence providing for application of relational theory
- As soon as all argument relations are aligned, temporal queries can be executed using relational operations



Computing a Value for a Period

- A single value of a column D for a business key (B) and a specified (query) period
- If the value do not change, return this value
- Must be equal for periods of same duration

$$D_q(B) = \sum_{BK(\text{row})=B} \frac{\text{duration}(P(\text{row}) \cap P(q))}{\text{duration}(P(q))} D_{\text{row}}$$



Accumulating Values over a Period

- Suitable for attributes like speed, rate, etc. (km/hour, salary/month, ...)
- If the value do not change, return this value multiplied by the duration of the query period

$$A_q(B) = \frac{\cancel{\text{duration}(P(q))}}{\text{duration}(\text{time unit})} \sum_{BK(\text{row})=B} \frac{\text{duration}(P(\text{row}) \cap P(q))}{\cancel{\text{duration}(P(q))}} D_{\text{row}}$$

$$D_q(B) = \sum_{BK(\text{row})=B} \frac{\text{duration}(P(\text{row}) \cap P(q))}{\text{duration}(P(q))} D_{\text{row}}$$

Computing Period Aggregates: PostgreSQL

Surprisingly, require single pass over aggregated data

```
create or replace
function duration (p_period tstzrange)
returns double precision
IMMUTABLE language SQL
return  extract(epoch from upper(p_period) -
lower(p_period));

--- sum
create or replace function  period_sum_next
(
    accum double precision,
    val double precision,
    val_period tstzrange,
    query_period tstzrange)
returns double precision
IMMUTABLE language SQL
```

```
return case
    when val_period && query_period
and val is not null
    then
        coalesce(accum,0) + val *
            duration (val_period *
                query_period)/
            duration (query_period)
    else accum
end;
drop AGGREGATE if exists  period_sum (
    double precision, tstzrange, tstzrange);
CREATE AGGREGATE  period_sum (
    double precision, tstzrange, tstzrange
) (
    sfunc=  period_sum_next,
    STYPE = double precision);
```

Example: count, avg, sum

- Number of employees is not an integer

```
psql> \set query_period $$[2024-04-01,2024-08-01)$$
psql> select
    period_count(salary, valid_at, :query_period)::decimal(9,2)  period_cnt,
    period_avg(salary, valid_at, :query_period)::decimal(9,2)    salary_avg,
    period_sum(salary, valid_at, :query_period)::decimal(9,2)    salary_sum
from   temporal_emp
where  valid_at && :query_period
;
 period_cnt | salary_avg | salary_sum
-----+-----+-----
          5.50 |      4209.99 |    23154.92
(1 row)
```

Example: period aggregates per project

- Accumulated values show person-months and total spendings

```
psql> \set query_period $$[2024-04-01,2024-08-01)$$
psql> select
    project,
    period_count(salary, valid_at, :query_period)::decimal(9,2) period_cnt,
    period_accum(1, make_interval(months => 1),
    valid_at, :query_period)::decimal(9,2) person_months,
    period_avg(salary, valid_at, :query_period)::decimal(9,2) period_avg,
    period_sum(salary, valid_at, :query_period)::decimal(9,2) period_sum,
    period_accum(1, make_interval(months => 1),
    valid_at, :query_period)::decimal(9,2) salary_accum
from temporal_emp
where valid_at && :query_period
group by project;
```

project	period_cnt	person_months	period_avg	period_sum	salary_accum
p11	1.12	4.57	3687.59	4140.98	4.57
p15	3.61	14.70	4330.39	15653.28	14.70
p20	0.76	3.10	4408.60	3360.66	3.10

(3 rows)

Multiple Periods

Periods are obtained from the DB

```
psql> select e.project, q.code,
  period_count(salary, e.valid_at, q.valid_at)::decimal(9,2) period_cnt,
  period_accum(1, make_interval(months => 1),
  e.valid_at, q.valid_at)::decimal(9,2) person_months,
  period_avg(salary, e.valid_at, q.valid_at)::decimal(9,2) period_avg,
  period_sum(salary, e.valid_at, q.valid_at)::decimal(9,2) period_sum,
  period_accum(e.salary, make_interval(months => 1),
  e.valid_at, q.valid_at)::decimal(9,2) salary_accum,
  q.valid_at
from temporal_emp e
join qu q on e.valid_at && q.valid_at
where q.valid_at && tstzrange('2024-01-01', '2025-01-01')
group by q.code, e.project, q.valid_at
order by e.project, q.code;
```

project	code	period_cnt	person_months	period_avg	period_sum	salary_accum	valid_at
p11	23.q1	1.66	5.03	4079.49	6770.50	20527.78	["2024-01-01 00:00:00+02", "2024-04-01 00:00:00+03"]
p11	23.q2	1.16	3.53	3713.21	4325.27	13120.00	["2024-04-01 00:00:00+03", "2024-07-01 00:00:00+03"]
p11	23.q3	0.34	1.03	3600.00	1213.04	3720.00	["2024-07-01 00:00:00+03", "2024-10-01 00:00:00+03"]
p15	23.q1	2.34	7.10	4584.56	10729.50	32531.25	["2024-01-01 00:00:00+02", "2024-04-01 00:00:00+03"]
p15	23.q2	3.82	11.60	4383.05	16761.54	50843.33	["2024-04-01 00:00:00+03", "2024-07-01 00:00:00+03"]
p15	23.q3	3.00	9.20	4165.94	12497.83	38326.67	["2024-07-01 00:00:00+03", "2024-10-01 00:00:00+03"]
p15	23.q4	3.00	9.20	4233.33	12700.00	38964.31	["2024-10-01 00:00:00+03", "2025-01-01 00:00:00+02"]
p20	23.q1	1.00	3.03	3200.00	3200.00	9702.22	["2024-01-01 00:00:00+02", "2024-04-01 00:00:00+03"]
p20	23.q2	0.34	1.03	4025.81	1371.43	4160.00	["2024-04-01 00:00:00+03", "2024-07-01 00:00:00+03"]
p20	23.q3	2.00	6.13	4697.83	9395.65	28813.33	["2024-07-01 00:00:00+03", "2024-10-01 00:00:00+03"]
p20	23.q4	2.00	6.14	4900.00	9800.00	30066.94	["2024-10-01 00:00:00+03", "2025-01-01 00:00:00+02"]

(11 rows)

Quick Intro
Getting Data from a Temporal Database
Integrity Constraints
Temporal Aggregation

Implementing the Temporal Algebra

*



Temporal Relational Algebra

Extending Relational Algebra with Periods

- Filtering: does not require extension
- Project: duplicate elimination + merge periods
- Inner join, intersect, and cartesian product: intersection of periods
- Outer join, semi-join, anti-join, union, difference: difference and union of periods may result in multiple periods

Left period			
Right period			
Left Join	(Left, NULL)	(Left, Right)	(Left, NULL

Temporal Query Processing

Implementation Strategy Alternatives

- Point-in-time based: Merge result for all points in (discrete) time
- Alignment operator: reducing temporal algebra to relational algebra
- Specify temporal operations as relational expressions
- Modified relational algebra algorithms (incorporate range algebra)

PostgreSQL: Algebra of Periods

Ranges and Multi-Ranges

- An intersection of periods is a period, but union and difference can be a set of periods
- The set of finite sets of periods is closed under intersection, union, and difference
- PostgreSQL: periods $\Rightarrow$ range (of timestamps), set of periods $\rightarrow$ multi range
- All operations on multirange types needed for temporal algebra are already implemented
- We associate multirange (instead of single period) to each row of the intermediate results in an algebraic expression representing a temporal query

Mapping temporal algebra to SQL

Using period algebra to produce multiple output rows from a single pair of input rows

- An example: temporal left outer join:

```
select l.attributes, r.attributes
      l.period * coalesce(r.period, l.period) as period
from left_table l
      left join right_table r
      on l.join_key = r.join_key
      and l.period && r.period
UNION all (
select l.attributes, NULL as r_attributes
      unnest (
        multirange(l.period) - range_agg(r.period)) as period
from left_table l join right_table r
      on l.join_key = r.join_key
      and l.period && r.period
group by l.attributes, l.period);
```

- Combines the alignment with join
- Does not require any modifications to run-time, compiler support is desirable

Quick Intro
Getting Data from a Temporal Database
Integrity Constraints
Temporal Aggregation
Implementing the Temporal Algebra
Conclusions



Conclusions

- The demand for support of time is high
- The demand is underestimated and is not properly supported by neither DBMSs nor application development methodologies
- A consistent implementation is feasible
- Several important features needed to support time are already in place

Thank You!